

大规模 Web Crawler 架构总结

面向 Parallel (Search Ranking) 面试准备。标注了哪些有公开文献支撑、哪些是推断、哪些来自实践经验。

一、单机调度器:正确的形状是事件驱动,不是查表

1.1 拉模型的陷阱

如果把调度器接口设计成同步查询:

```
next_ready(t) -> url | None
```

即"外部喂一个时刻 t,问此刻该发哪个 URL",就会凭空制造出一个难题:

- 主序要求按 arrival 最早
- 可发条件依赖 `host_last_sent + delay <= t`

这是两个独立维度,无法压进单个有序结构。硬做需要两个堆(一个按 arrival、一个按 ready_time)、堆间状态迁移、懒删除,还要额外假设 t 单调不减(否则迁移不可逆,退化成全扫)。

这些难点全部是接口造成的,不是爬虫调度固有的。

1.2 事件驱动:真实形状

真实系统是两组多路复用,事件被推过来,不需要查找:

- K 个 fd 谁 ready — 由 epoll/select(内核)管
- N 个 host 谁到期 — 由 timer(delay queue / timing wheel)管

单机状态:

结构	内容	作用
m1	host -> {URL 队列, delay, last_sent}	per-host 真相
timer	按 ready_time 排的最小堆	谁何时解冻
ready deque	已变绿、待发出的 URL	FIFO 取队头
连接池	K 个连接槽	并发上限

三个事件入口:

1. **add** — 新 URL 入 m1;若该 host 空闲,直接进 ready deque
2. **timer 到期** — host 变绿,从 m1 取队头 URL 入 ready deque
3. **连接空闲** — 从 ready deque 取一个发出;该 host 按 $t + \text{delay}$ 重新挂 timer

1.3 三个消失的东西

- **inflight < K 的显式判断消失** — K 变成资源池大小。没有空闲槽就没有取的动作,超限在结构上不可能发生,而不是靠一个可能写错的判断守住。
- **complete() 接口消失** — politeness 只依赖发出时刻($\text{ready_time} = \text{发出时刻} + \text{delay}$),与请求何时返回无关。释放并发额度是 I/O 事件的内部回调,不是外部 API。
- **ready 集合的排序需求消失** — host 变绿的顺序天然就是时间顺序(timer 按到期时刻排),所以 ready 集合可以是 deque 而非堆,取队头 O(1)。用双端而非单端,是为了让重试/高优先级 URL 从头部插队。

拉模型下之所以需要按 arrival 排堆,是因为所有 host 被同时"发现"可发,先后关系丢失了,只能靠 arrival 重建。推模型下 timer 保留了这个顺序。

二、Frontier:抓什么、按什么顺序

2.1 不是 DFS

种子列表 → 抓取 → 抽链 → 新 URL 入 frontier → 迭代。形状接近 BFS,但真正决定顺序的是优先级,不是遍历策略。

2.2 排序键

真实的 frontier 排序键:

- PageRank / 站点质量
- 上次抓取时间(新鲜度)
- 预估更新频率
- 历史状态:404、5xx、robots 拒绝

注意:全局 arrival FIFO 在 web crawl 里没有实际价值。 面试题里出现"按到达顺序公平"这类 tie-break 规则,通常是为了让题目可判定而人为加的,不是工业实践。

2.3 Mercator 的双层队列

- **front queues** 管优先级
- **back queues** 管 politeness

优先级和 politeness 是两个正交关注点,分两层处理。

三、分布式:分片键的选择

3.1 结论:按 host 分片,不是按 URL

这是有明确文献支撑的教科书结论:

- **Manning IR 教科书**把"为什么按 host 而非单个 URL 分区更好"直接作为习题
- **Mercator / Atrax**:分布式配置下主机名空间在进程间划分,每个进程只负责自己那个子集;后续版本按主机名划分 URL 空间,并避开中心化 URL 服务器这个瓶颈
- **UbiCrawler**:用一致性哈希按 URL 的 host 部分分片
- **Compaq 分布式爬虫专利**:按主机名分配使 politeness 更容易实施,且避免在多个爬虫间复制每主机信息
- **BUBiNG**:UbiCrawler 作者的后续系统,完全分布式,有公开论文

3.2 为什么不按 URL hash

按 URL hash 会把同一 host 的 URL 打散到所有机器,后果:

1. **politeness 守不住** — 每台机器都以为自己在守 10 QPS,100 台合起来打出 1000。必须引入分布式 quota 来打补丁。
2. **per-host 状态无处安放** — robots.txt(及其 24h 缓存)、DNS 解析结果、crawl-delay、host 历史响应状态。要么 N 台各存一份(浪费且不一致),要么跨机查(延迟)。这条和 rate limit 无关,即使有完美的中心 limiter 也存在。
3. **缓存局部性丧失** — Manning 明确指出分布式爬取的关键目标是 cache locality。
4. **静态均分额度的利用率问题** — 某 host 的 URL 常常是阵发性的:一轮扩边挖出 5000 个,之后长期沉寂。均分到 100 台后,多数机器额度闲置,有活的那台被卡在 1/N,实际吞吐远低于 host 允许的上限。要修就得让额度流动,而额度流动需要中心协调 —— 正是 Mercator 明确要避开的东西。

3.3 host 分片的代价(真实难点)

- **负载倾斜** — 少数超大 host 的 URL 量可能是小 host 的百万倍
- **rehash 迁移** — 机器增减时,迁移期间 politeness 短暂失效。一致性哈希缓解但不消除
- **hostname vs IP 的粒度问题** — Compaq 专利指出:按符号主机名分发时,两个不同主机名可能解析到同一 IP,落到两台不同机器上,于是并行打同一台物理服务器;按 IP 分发才能保证只有一个爬虫访问该 IP。共享主机场景下这是真实漏洞。

3.4 三套分片键并存

一个真实爬虫里至少有三个不同的分片键:

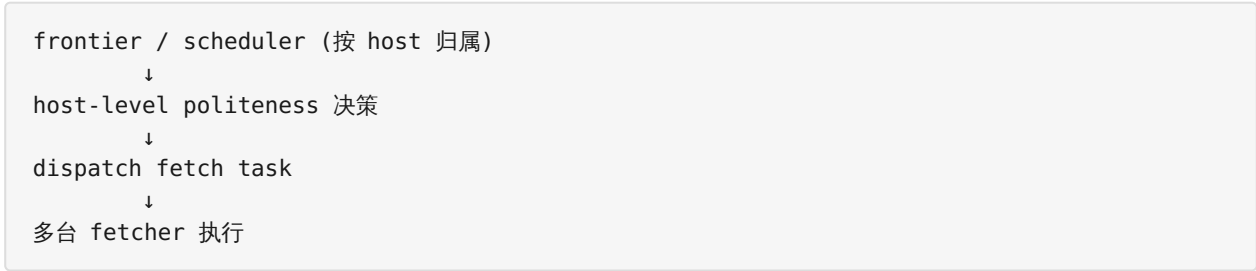
组件	分片键	原因
URL frontier	host	politeness + per-host 状态局部性
URL 去重(Bloom filter)	URL hash	不需要 per-host 状态
内容去重(指纹/shingle)	指纹属性	相同内容会出现在不同 host 上,无法按 host 分片

最后一条是 Manning 明确点出的:内容指纹不能按主机名分区,因为没有什么能阻止相同或高度相似的内容出现在不同的 web server 上。

四、调度归属 vs 物理抓取

一个重要澄清:"同一 host 由同一台机器负责"严格讲是指调度决策由单一决策点做出,不必然是同一台机器执行 fetch。

理论上可以是:



这个解耦在特定场景下是必要的:

- fetch 需要地理分散(从不同区域出口访问)
- fetch 是重资源的(需要浏览器渲染 JS)
- fetcher 侧有独立的扩缩容需求

但它不是通例。 公开文献里的大规模系统(Mercator、UbiCrawler、Manning 的 host splitter)都是 host 亲和到物理节点。解耦的代价是每个 URL 都要过一次中心决策点,重新引入了 Mercator 要避开的瓶颈(可以用批量令牌/租约摊薄,但不免费),而且 3.2 里的 per-host 状态问题依然存在。

准确的表述:**host 分片是默认;调度与抓取解耦是当 fetch 侧有独立约束时的演化,不是取代关系。**

五、超大 host 的分层处理

不要为 100% 的 host 付协调成本,只为 0.01% 付。

层	对象	策略
长尾	绝大多数 host	hash 到单机, politeness 纯本地, 零协调。一台机器管数万个小 host
巨头	极少数超大 host	专用小集群, 集群内共享 host 级配额

分界线应该是动态的, 不是静态名单: frontier 积压超阈值、或单机额度长期打满, 就提升到集群模式。web 的 host 规模分布随时间变化, 静态名单会过期。

集群内的配额分配 —— 两种做法, 适用条件不同:

- **静态均分**: 目标 host 的 URL 量足够大且持续时(单站定向深度抓取), 每台机器随时有活干, 额度不会闲置。**实践中够用, 加协调只是徒增复杂度**。(来自 Pinterest 抓取的实践经验)
- **租约/批量令牌**: URL 阵发性到达时才需要。limiter 一次发 N 个配额 + 时间窗, 机器在窗内自用, 用完再要。中心 QPS 从"每次抓取一次"降到"每 N 次一次", 代价是窗口边界可能短暂超额, 靠配额设保守吸收。

一个取舍不是纯技术问题: 超大 host 往往同时是 robots.txt 最严、最容易封 IP 的。给它们配集群不只为吞吐, 也为从多出口 IP 分散访问 —— 但这又与 politeness 的精神冲突(取决于对方按 IP 还是按 UA 识别)。

六、监控

6.1 核心信号

- **per-host 实际 fetch rate vs 分配额度**
- **全局 fetch QPS** —— 系统健康总闸

低于阈值即报警。

6.2 必须能区分的两类原因

单看 fetch rate 无法定位问题, 要配合其他指标:

现象	组合信号	判定
额度没用满 + frontier 非空	—	真问题(DNS 慢、连接池耗尽、目标站变慢)
额度没用满 + frontier 空	—	正常, 没活干
fetch rate 下降	HTTP 状态码分布(429 占比)、响应体大小分布	目标站限速/降级, 而非本系统故障

第二类容易误判成自己系统的问题然后去调错的地方。

6.3 顺带的用途

per-host 的"额度利用率"本身就是**动态额度调整的输入**:某台持续用不满而另一台持续打满 → 分配失衡 → 挪额度。这比租约机制简单,且基于观测而非每次请求协调。

七、公开材料的边界

系统	公开程度
Mercator / Atrax	有论文(Najork),最经典的参考
UbiCrawler / BUBiNG	有论文,一致性哈希 host 分片
Manning IR 教科书第 20 章	完整架构 + host splitter + 三套分片键
Compaq 分布式爬虫专利	分片键选择的理由写得最直白,含 hostname vs IP
Googlebot	黑盒。只有 1998 年 Brin & Page 那篇(中心化 URLserver + 约 300 并发连接 + 四台机器约 100 页/秒),该架构早已作废。现代版本只有 Search Central 文档和公开发言里的行为描述
Bingbot	黑盒。没有微软发表的系统设计文献,只有 SEO 侧的行为描述

面试时引 Mercator 比引 Google 安全 —— 前者有论文,后者只能猜。被问到 Google/Bing 时直接说"没有公开架构文献,只有行为层面的观察",比编要好。

八、可直接复用的判断

1. 接口形状决定难度。拉模型 `next_ready(t)` 制造的两维排序冲突,在推模型下不存在。识别出"这个抽象把好问题扭曲成了坏问题"本身是有价值的判断。
2. 常数因子可以压过渐进复杂度。host 数量比 URL 少几个数量级(亿级 URL / 百万级 host,活跃 host 更少)。O(H) 的连续内存扫描配合 cache 友好性,很可能快过两个堆 + 懒删除 + 迁移的 O(log H),而且代码量少一个数量级、没有不变式要维护、不需要时间单调性假设。知道什么时候不该优化。
3. 把协调成本花在例外上,不是默认路径上。